

Experiment No.	9
Experiment Title	Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning
Student Name	Hemnath D
Register Number	3122247001022
Course	M.Tech Integrated Computer Science and Engineering
Faculty	Dr. Poreddy Ajay Kumar Reddy
Date of Submission	19 September 2026

1 Objective

The aim of this experiment is to implement and compare two neural classifiers on the English Handwritten Characters dataset: Model A, a single-layer Perceptron Learning Algorithm (PLA) written from scratch with a step activation and the classic weight update rule, and Model B, a Multilayer Perceptron (MLP) with hidden layers and non-linear activations trained by backpropagation. The MLP's activation function, optimizer, learning rate, batch size, number and size of hidden layers, cost function and regularization are selected through systematic tuning, and the two models are compared with accuracy, precision, recall, F1-score, confusion matrices, micro and macro averaged ROC curves and training convergence curves.

2 Problem Statement

The task is 62-class image classification: given a scanned handwritten character (a digit 0–9, an uppercase letter A–Z or a lowercase letter a–z), predict which character it is. The input is the flattened, normalised 32×32 grayscale image (1,024 features) and the output is one of 62 labels. With only 55 images per class the problem is small but hard: many classes are near duplicates of each other (0/O/o, 1/I/l, C/c, S/s, Z/z), and handwriting varies in slant, stroke width and size. The experiment asks whether a linear model can handle this, how much a hidden layer adds, and which hyperparameters actually matter when training the MLP.

3 Dataset Description

Dataset Name	English Handwritten Characters Dataset
Dataset Source	Kaggle (dhruvildave/english-handwritten-characters-dataset), derived from the Chars74K handwritten subset
Number of Samples	3,410 images (1200×900 RGB scans, one character each)
Number of Features	1,024 after preprocessing (32×32 grayscale pixels scaled to $[0, 1]$)
Number of Classes	62 (digits 0–9, uppercase A–Z, lowercase a–z), exactly 55 images per class
Missing Values	None; every path in <code>english.csv</code> exists on disk and there are no duplicate file names

Train-Test Split	Stratified 80/20 split into 2,728 train+validation and 682 test images (11 per class); the 2,728 were further split into 2,294 training (37 per class) and 434 validation (7 per class) images
------------------	--

4 Exploratory Data Analysis

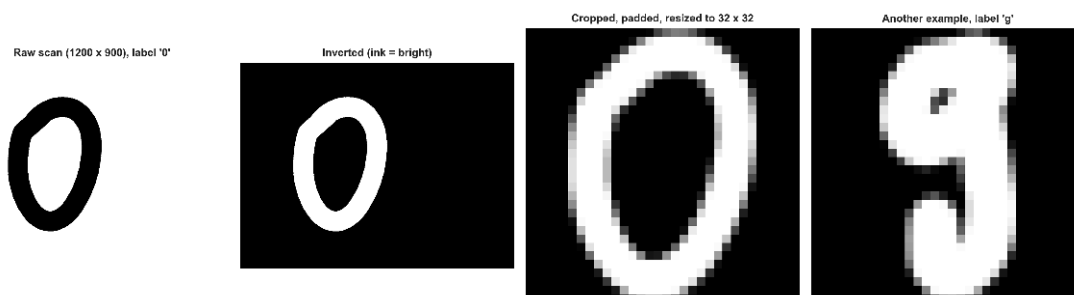


Figure 1: A raw scan, its inverted version, the cropped and resized 32×32 result, and a second example.

Inference: The raw scans are large (1200×900) with the character occupying only the middle 40% of the frame and a lot of white background. After inversion, cropping to the ink bounding box with padding, and resizing, the character fills the 32×32 frame with a consistent size and position. This removes the position and scale variation that would otherwise dominate the pixel features and reduces the input from 1.08 million values to 1,024.

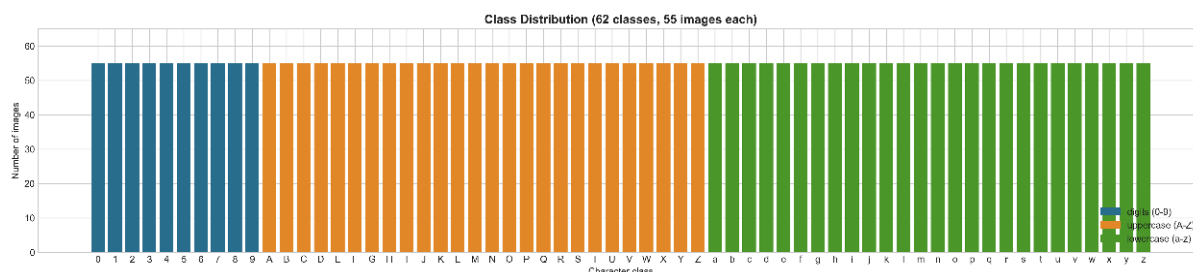


Figure 2: Number of images per class (blue digits, orange uppercase, green lowercase).

Inference: The dataset is perfectly balanced with 55 images in each of the 62 classes, so accuracy is a fair headline metric and macro and weighted averages of precision, recall and F1 coincide. The downside is that 55 images per class is very little for a 1,024-dimensional input, which is the root cause of the overfitting seen later.

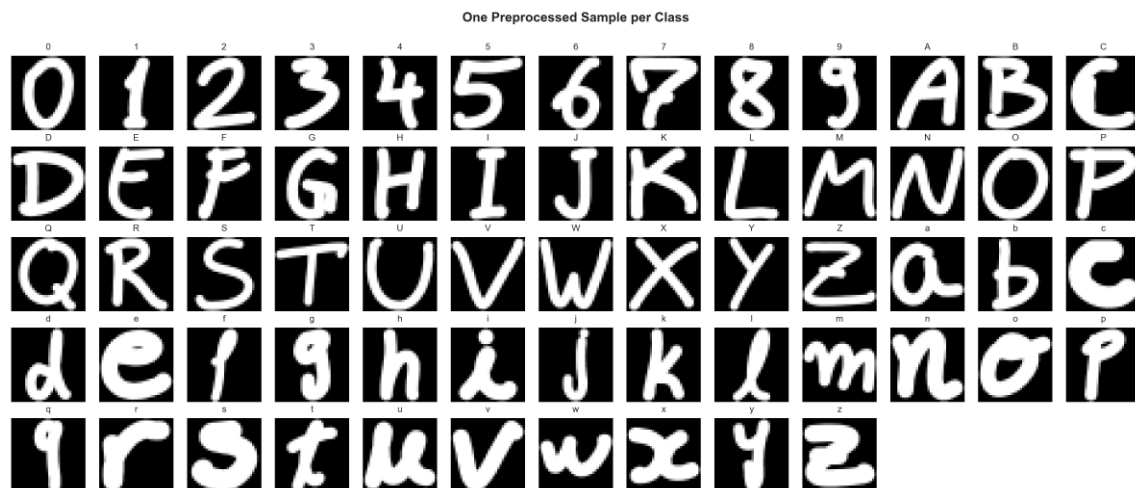


Figure 3: One preprocessed sample from every class.

Inference: The samples show the ambiguity built into the label set. Uppercase and lowercase C, O, S, V, W, X and Z differ only in size in the original scan, and that difference is removed by the crop-and-resize step. Likewise 0 and O, and 1, I and l, are visually the same stroke. Any classifier that only sees the normalised image will confuse these pairs, so a test accuracy well below 100% is expected even for a good model.

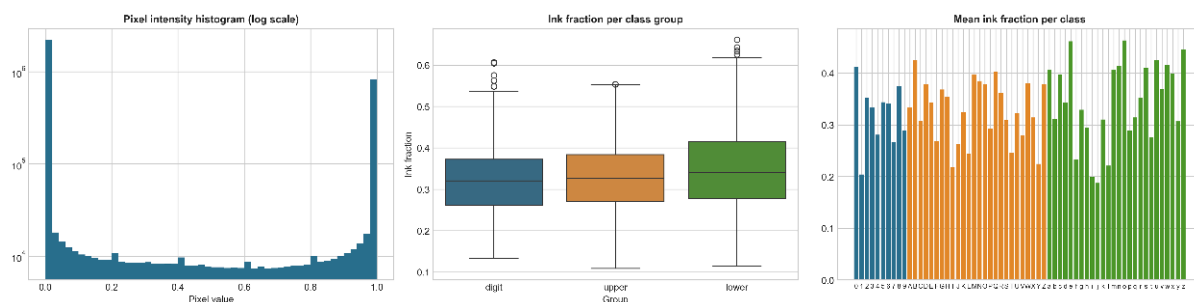


Figure 4: Pixel intensity histogram, ink fraction per class group and mean ink fraction per class.

Inference: Pixel values are strongly bimodal (mostly background near 0, ink near 1) with anti-aliased edges in between. Roughly a third of each image is ink (mean ink fraction 0.32 for digits, 0.33 for uppercase, 0.35 for lowercase). Simple statistics such as ink fraction vary more within a class than between classes, so the classifiers need the spatial layout of the pixels, not summary statistics.

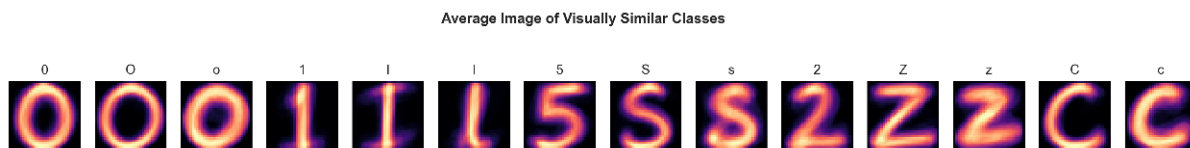


Figure 5: Average image of visually similar classes.

Inference: The average images of 0, O and o, of 1, I and l, and of the C/c, S/s and Z/z pairs are almost indistinguishable. These are exactly the pairs that dominate the confusion matrices of both models later, and they set a ceiling on the attainable accuracy.

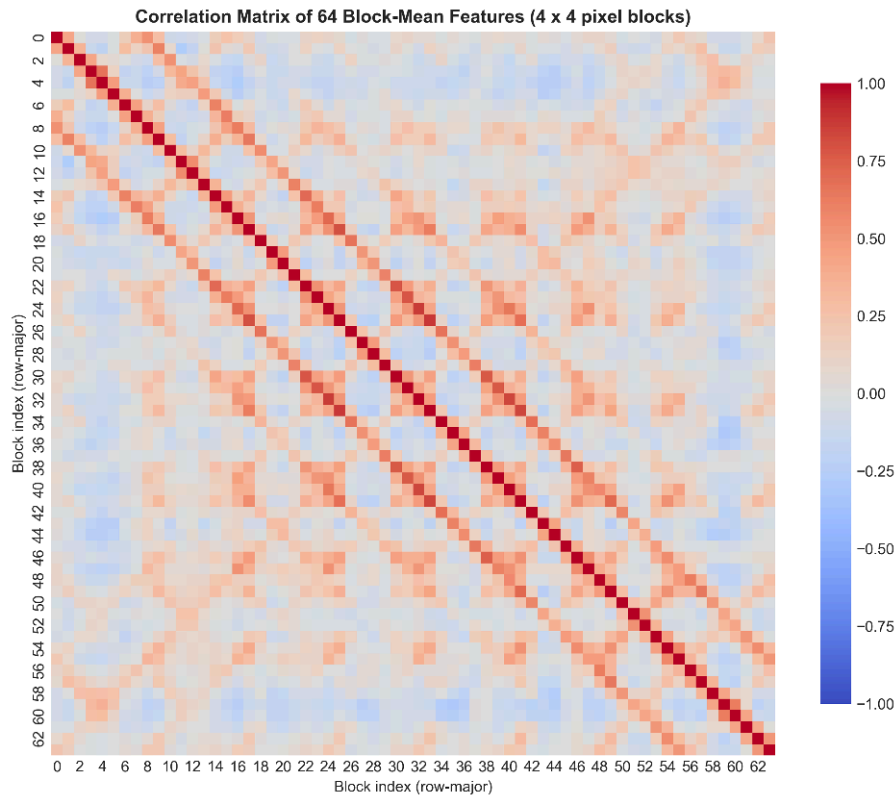


Figure 6: Correlation matrix of 64 block-mean features (each block is 4×4 pixels, row-major order).

Inference: Correlation is concentrated near the diagonal and in bands offset by 8 (vertical neighbours), so neighbouring blocks are correlated but distant ones are not (only 3.4% of pairs have $|r| > 0.5$, mean $|r| = 0.14$). The image features are locally redundant but globally diverse, which is why a fully connected hidden layer can learn useful stroke detectors from raw pixels.

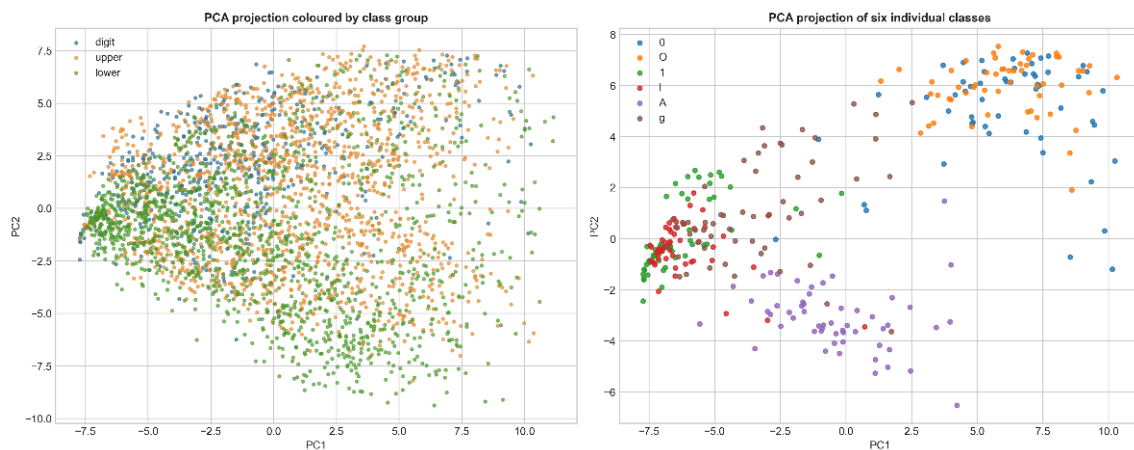


Figure 7: PCA projection of all images coloured by class group (left) and of six individual classes (right).

Inference: The first two principal components explain only 19.4% of the variance (50 components are needed for 85.8%), and the three class groups sit on top of each other. Even individual classes such as 0 and O or 1 and l overlap completely, while A and g are only partly separated. The classes are therefore not linearly separable in pixel space, which is the theoretical

reason the PLA is expected to struggle.

5 Data Preprocessing

- **Missing value handling:** No missing images or labels; every listed file exists.
- **Image preprocessing (resize, flatten, normalize):** Each scan was converted to grayscale, inverted so that ink is bright, thresholded at 50 to find the ink bounding box, placed on a square black canvas with 8 pixels of padding so that the aspect ratio is preserved, resized to 32×32 with area interpolation, scaled to $[0, 1]$ and flattened to a 1,024-vector.
- **Encoding:** The 62 character labels were integer coded with `LabelEncoder` (0–9, A–Z, a–z in order). The PLA uses a one-hot target vector per sample and the MLP uses the integer index with cross-entropy (or one-hot with MSE).
- **Feature scaling:** Pixel values are already in $[0, 1]$ after normalisation; no further scaling was applied.
- **Train-validation-test split:** Stratified 80/20 split (2,728 / 682), then a stratified split of the 2,728 into 2,294 training and 434 validation images. The validation set was used for hyperparameter selection and early stopping of both models, and the test set was used once at the end. Both models were trained on exactly the same 2,294 images.
- **Data augmentation (evaluated, not selected):** Random rotation ($\pm 12^\circ$), scaling (0.9–1.1) and translation (± 2 px) applied freshly every epoch was tried as a regularizer in the last tuning stage.

6 Machine Learning Algorithms

6.1 Model A: Perceptron Learning Algorithm

A perceptron outputs $\hat{y} = \text{step}(\mathbf{w}^\top \mathbf{x} + b)$, firing 1 when the net input is positive and 0 otherwise. For 62 classes one perceptron per class was trained one-vs-rest, giving a 62×1025 weight matrix (1,024 pixels plus a bias). Each training sample is shown to all 62 units, the one-hot target is compared with what fired, and every wrong unit is corrected with

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \eta(y - \hat{y}) \mathbf{x}.$$

The predicted class is the unit with the largest net input. Because the data is not linearly separable the updates never stop, so the weights from the epoch with the best validation accuracy were kept. The PLA is simple, quick (3.3 s for 60 epochs in NumPy) and easy to inspect, and it is guaranteed to converge if the classes are linearly separable. Its problems are the flip side: it can only draw straight boundaries, it does not settle on non-separable data, it has no idea of margin so the boundary it finds generalises badly, and the step function has no gradient, so it cannot be stacked into deeper networks.

6.2 Model B: Multilayer Perceptron

The MLP passes the input through one or more hidden layers, $\mathbf{h}_l = \phi(\mathbf{W}_l \mathbf{h}_{l-1} + \mathbf{b}_l)$, with a non-linear ϕ (ReLU, tanh or sigmoid), and a 62-unit output layer whose softmax gives class probabilities. Training minimises the cross-entropy $L = -\sum_c y_c \log p_c$ (or the squared error between the softmax output and the one-hot target) with backpropagation: the chain rule gives the gradient of the loss for every weight, layer by layer, and an optimizer moves the weights against it. Plain SGD steps by $\eta \nabla L$, momentum keeps a running velocity so steps build up in consistent directions, and Adam rescales each parameter's step by running estimates of its

gradient’s mean and variance. The hidden layer is what lets the MLP learn curved boundaries and features shared across classes, which is its whole advantage over the PLA. In return it brings many interacting hyperparameters, a non-convex loss where different seeds give different answers, a strong tendency to overfit small datasets, and a higher cost per epoch.

7 Experimental Setup

Python Version	3.13.5
Key Libraries	PyTorch 2.10.0 (MLP), NumPy (PLA), scikit-learn 1.7.2 (splits, metrics), OpenCV 4.12, pandas, Matplotlib, Seaborn
IDE	Jupyter Notebook / VS Code
Operating System	macOS 26.6.2
Hardware	Apple M3 Pro, 18 GB RAM; all training on CPU (10 threads)

8 Hyperparameter Selection

8.1 PLA

The PLA has only the learning rate and the number of epochs. With zero initial weights it is exactly invariant to η : scaling η scales every weight by the same factor and changes neither the sign of the net input (the step) nor the argmax, so $\eta = 0.01, 0.1, 1$ and 10 gave identical error curves (train error 0.0615 after 20 epochs, best validation accuracy 68.66% at epoch 18) and only the mean weight magnitude changed (0.05 to 52). $\eta = 1$ and 60 epochs were used, and the weights from the best validation epoch (38) were kept.

8.2 MLP: staged tuning

Every configuration was trained with three different seeds for up to 60 epochs with early stopping (patience 10) on the validation error, and the mean validation accuracy over the seeds was used to select each setting in turn. The baseline was one hidden layer of 256 units, ReLU, Adam with learning rate 0.001, batch size 64 and cross-entropy.

Table 2: Stage 1: activation function (mean $\pm$ std of validation accuracy over 3 seeds)

Activation	Val. accuracy	Train accuracy	Best epoch	Gap
ReLU	0.7972 $\pm$ 0.0050	0.9696	21.7	0.172
tanh	0.7965 $\pm$ 0.0047	0.9788	24.3	0.182
sigmoid	0.7888 $\pm$ 0.0115	0.9337	25.3	0.145

Table 3: Stage 2: optimizer and learning rate (3 seeds)

Optimizer	Learning rate	Val. accuracy	Train accuracy	Best epoch	Time (s)
SGD	0.01	0.7296 ± 0.0078	0.7700	51.7	1.10
SGD	0.05	0.7719 ± 0.0068	0.8612	31.0	0.82
SGD	0.1	0.7811 ± 0.0132	0.9236	29.7	0.82
SGD	0.3	0.8018 ± 0.0065	0.9653	23.7	0.71
SGD + momentum 0.9	0.005	0.7880 ± 0.0075	0.9092	44.3	1.25
SGD + momentum 0.9	0.01	0.7865 ± 0.0054	0.9000	23.3	0.77
SGD + momentum 0.9	0.05	0.8065 ± 0.0019	0.9927	29.0	0.80
SGD + momentum 0.9	0.1	0.7911 ± 0.0076	0.9753	19.7	0.61
Adam	0.0001	0.7642 ± 0.0043	0.8864	55.7	2.02
Adam	0.0005	0.7896 ± 0.0066	0.9638	28.0	1.18
Adam	0.001	0.7972 ± 0.0050	0.9696	21.7	0.96
Adam	0.005	0.7849 ± 0.0039	0.9725	15.0	0.74

Table 4: Stage 3: batch size (SGD + momentum, lr 0.05, 3 seeds)

Batch size	Val. accuracy	Train accuracy	Best epoch	Time (s)
16	0.5998 ± 0.0093	0.6915	15.0	1.93
32	0.8126 ± 0.0071	0.9942	30.3	1.57
64	0.8065 ± 0.0019	0.9927	29.0	0.96
128	0.7949 ± 0.0019	0.9705	21.3	0.50
256	0.7796 ± 0.0022	0.8806	16.3	0.32

Table 5: Stage 4: hidden layers and neurons (3 seeds)

Hidden layers	Parameters	Val. accuracy	Train accuracy	Best epoch	Time (s)
(128)	139,198	0.7849 ± 0.0207	0.9741	23.7	1.00
(256)	278,334	0.8126 ± 0.0071	0.9942	30.3	1.44
(512)	556,606	0.8103 ± 0.0057	0.9711	13.7	1.09
(1024)	1,113,150	0.8103 ± 0.0022	0.9799	21.7	2.98
(256, 128)	303,294	0.7581 ± 0.0050	0.9108	19.0	1.21
(512, 256)	672,062	0.7957 ± 0.0217	0.9725	25.3	2.35
(512, 256, 128)	697,022	0.7412 ± 0.0054	0.8750	20.3	2.21
(256, 256, 256)	409,918	0.7512 ± 0.0050	0.8845	18.0	1.68

Table 6: Stage 5: cost function, and Stage 6: regularization (3 seeds, up to 100 epochs, patience 15)

Stage	Setting	Val. accuracy	Train accuracy	Gap
Loss	Cross-entropy	0.8126 ± 0.0071	0.9942	0.182
Loss	MSE on softmax	0.7911 ± 0.0029	0.9393	0.148
Regularization	none	0.8134 ± 0.0068	0.9939	0.181
Regularization	dropout 0.2	0.7873 ± 0.0029	0.9730	0.186
Regularization	dropout 0.4	0.7089 ± 0.0093	0.8330	0.124
Regularization	weight decay 10^{-4}	0.8157 ± 0.0050	0.9910	0.175
Regularization	weight decay 10^{-3}	0.7980 ± 0.0066	0.9823	0.184
Regularization	dropout 0.2 + wd 10^{-4}	0.7919 ± 0.0060	0.9661	0.174
Regularization	dropout 0.4 + wd 10^{-4}	0.7243 ± 0.0228	0.8333	0.109
Augmentation	none	0.8157 ± 0.0050	0.9910	0.175
Augmentation	rotation/scale/shift each epoch	0.8049 ± 0.0132	0.8633	0.058

Table 7: Final chosen hyperparameters

Parameter	Value	Justification
Hidden layers	1 layer, 256 units	Best validation accuracy; wider layers did not help and deeper ones hurt
Activation	ReLU	Marginally best and fastest to converge; tanh equivalent, sigmoid worse
Optimizer	SGD, momentum 0.9	Best mean validation accuracy with the smallest seed-to-seed spread (0.2%)
Learning rate	0.05	Best of 0.005–0.1 for momentum SGD; 0.1 became noisy, 0.01 was slow
Batch size	32	Best accuracy; 16 was unstable at this learning rate, 128+ underfit within the epoch budget
Loss	Cross-entropy	2.2 points better than MSE, faster loss decrease
Regularization	Weight decay 10^{-4} , no dropout	Dropout underfit with only 37 images per class; small weight decay gave a slight gain
Data augmentation	Not used	Cut the train/val gap from 17.5 to 5.8 points but did not raise validation accuracy
Epochs	Up to 150, patience 20, early stopping	Restores the weights of the best validation epoch (66 in the final run)

9 Results

Table 8: 5-fold cross-validation of the final MLP configuration on the 2,728 train+validation images

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean $\pm$ std
Accuracy	0.7619	0.7656	0.7949	0.7780	0.7431	0.7687 ± 0.0192
Best epoch	14	22	37	25	11	

Table 9: A/B comparison on the 682-image test set

Metric	PLA	MLP (untuned baseline)	MLP (tuned)
Accuracy	0.6437	0.7229	0.7317
Precision (macro)	0.6651	0.7388	0.7492
Recall (macro)	0.6437	0.7229	0.7317
F1-score (macro)	0.6395	0.7218	0.7305
F1-score (weighted)	0.6395	0.7218	0.7305
ROC-AUC (micro)	0.9588	0.9858	0.9841
ROC-AUC (macro)	0.9600	0.9872	0.9847
Training accuracy	0.9908	0.9738	0.9956
Validation accuracy	0.7120	0.7880	0.8272
Parameters	63,550	278,334	278,334
Training time (s)	3.3	0.9	3.8

Table 10: Mean F1-score per class group and the most frequent confusions of each model on the test set

Group	PLA F1	MLP F1	Most frequent confusions (true $\rightarrow$ predicted, count)
Digits (10)	0.6675	0.7292	PLA: 0 $\rightarrow$ O (4), I $\rightarrow$ 1 (4), o $\rightarrow$ 0 (4); MLP: I $\rightarrow$ 1 (5), o $\rightarrow$ 0 (4), g $\rightarrow$ 9 (4), 1 $\rightarrow$ l (4)
Uppercase (26)	0.6905	0.7648	PLA: C $\rightarrow$ c (7), W $\rightarrow$ w (5), S $\rightarrow$ s (4), X $\rightarrow$ x (3); MLP: Z $\rightarrow$ z (4), p $\rightarrow$ P (4), C $\rightarrow$ c (3)
Lowercase (26)	0.5779	0.6968	PLA: e $\rightarrow$ c (4), l $\rightarrow$ I (3); MLP: v $\rightarrow$ V (4), l $\rightarrow$ 1 (3), w $\rightarrow$ W (3)

The tuned MLP beats the PLA on every metric: 73.17% versus 64.37% accuracy, macro F1 0.731 versus 0.640 and macro ROC-AUC 0.985 versus 0.960. The gain is spread over all three class groups and is largest for lowercase letters. The untuned MLP already reached 72.29%, so most of the gap to the PLA comes from the hidden layer itself; tuning added 3.9 points of validation accuracy but only 0.9 points on the test set, which is within the sampling noise of a 682-image test set. The 5-fold cross-validation estimate of $76.9\% \pm 1.9\%$ for the final configuration sits between the validation and test scores and confirms that the model generalises at roughly three quarters accuracy.

10 Result Visualization

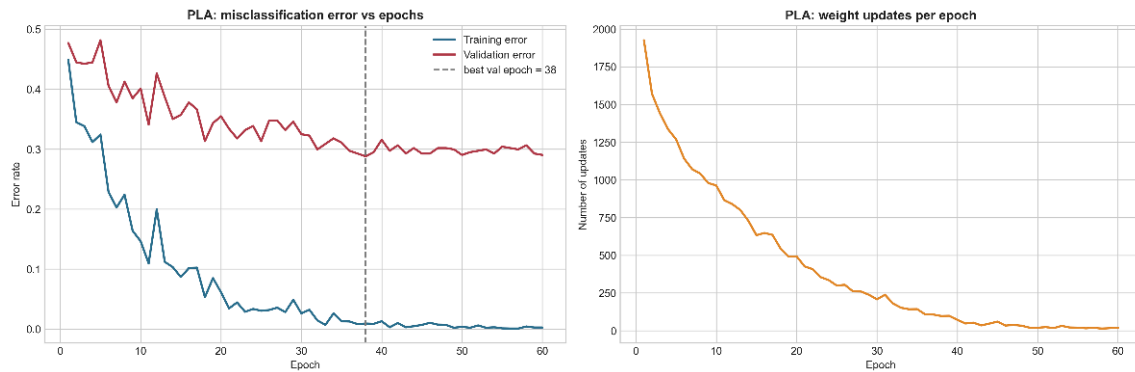


Figure 8: PLA training and validation error per epoch (left) and number of weight updates per epoch (right).

Inference: The training error falls steadily from 45% to 0.3% over 60 epochs because 2,294 samples in 1,025 dimensions are almost linearly separable, but the validation error stalls at about 30% after epoch 30 and oscillates. The number of updates per epoch never reaches zero (19 updates in the last epoch), which shows that the data is not separable and the PLA does not converge in the formal sense; the pocket weights from epoch 38 were used.

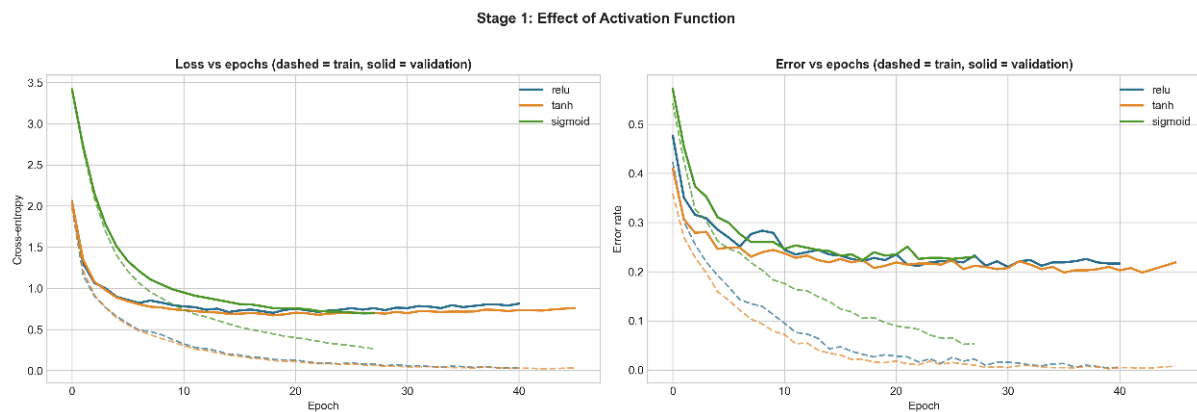


Figure 9: Stage 1: loss and error curves for ReLU, tanh and sigmoid.

Inference: All three activations reach a similar validation error of about 20%. ReLU and tanh converge at the same rate while sigmoid is a little slower and has a larger seed-to-seed spread, because its saturating output shrinks the gradients. The activation function was the least influential hyperparameter.

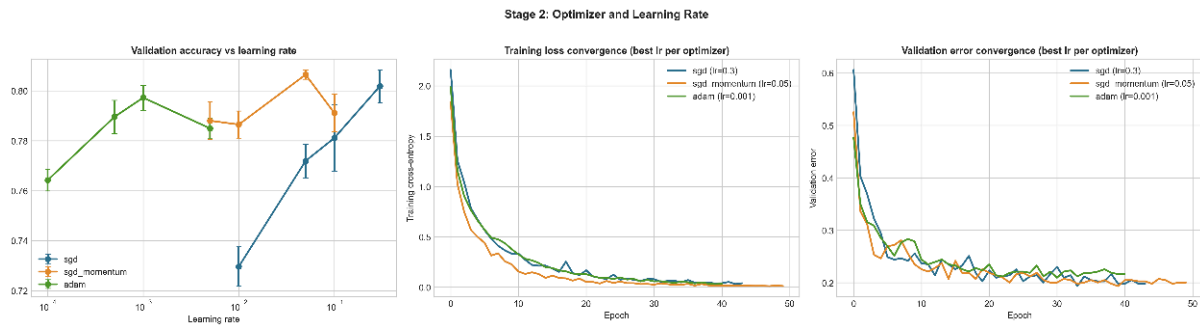


Figure 10: Stage 2: validation accuracy versus learning rate for each optimizer (left) and convergence of the best learning rate per optimizer (centre and right).

Inference: Adam is the most forgiving optimizer: its accuracy stays between 76% and 80% over a 50-fold learning rate range, whereas plain SGD climbs from 73% to 80% and needs a learning rate of 0.3, thirty times larger than Adam’s. With their best learning rates all three reach a similar loss, but momentum SGD drives the training loss down fastest and ends with the lowest validation error (80.65%), so it was chosen.

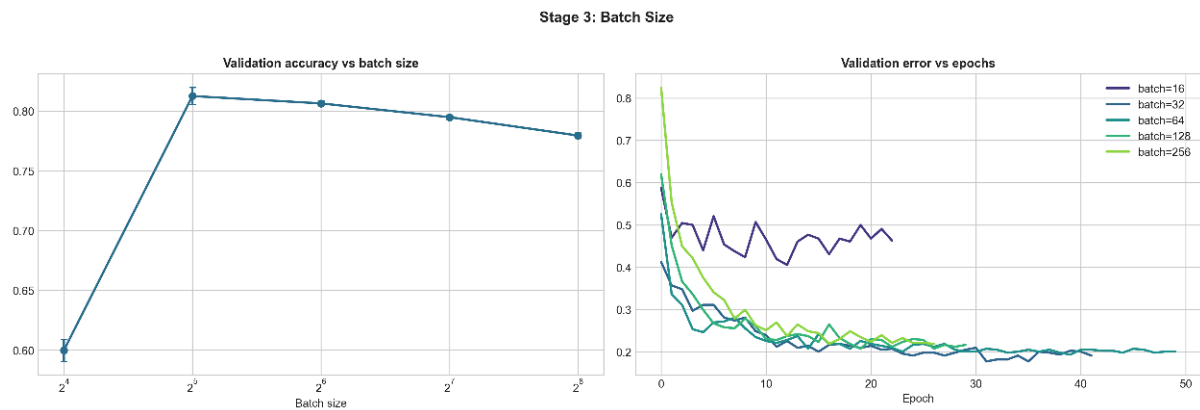


Figure 11: Stage 3: validation accuracy versus batch size and validation error curves.

Inference: Batch 16 combined with the 0.05 learning rate and momentum was unstable: the training loss bounced between 1.2 and 2.1 and the validation accuracy never exceeded 60%, because the gradient noise of 16-sample batches is too large for that step size. Batch 32 and 64 were best (81.3% and 80.7%), while 128 and 256 make fewer updates per epoch and stopped short of fitting the data within the epoch budget.

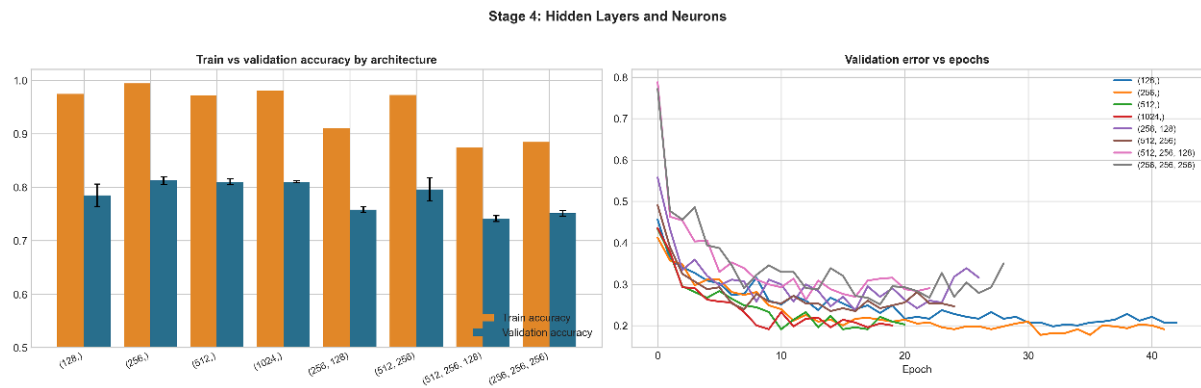


Figure 12: Stage 4: train and validation accuracy for each architecture and their validation error curves.

Inference: A single hidden layer of 256 to 1,024 units gives 81% validation accuracy regardless of width, while every deeper network is worse, down to 74.1% for three layers. The deeper networks also reach lower training accuracy (87% to 91%) and stop earlier, so they are harder to optimise with this learning rate as well as more prone to overfitting with only 2,294 images.

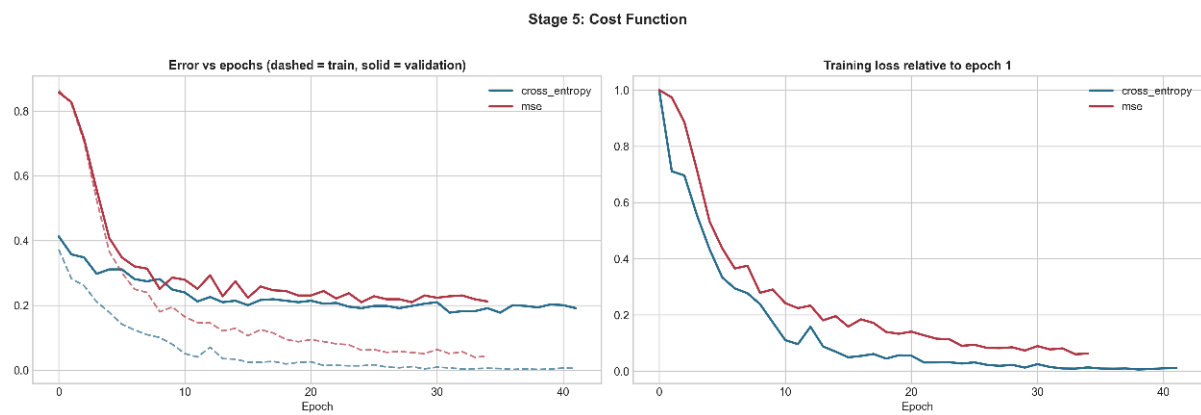


Figure 13: Stage 5: error and relative training loss for cross-entropy and MSE.

Inference: Cross-entropy drives the training error down faster and reaches 2.2 points higher validation accuracy than MSE on the softmax output. MSE saturates when the softmax is confident, so its gradients on hard examples are small, which slows learning and leaves the training accuracy at 94%.

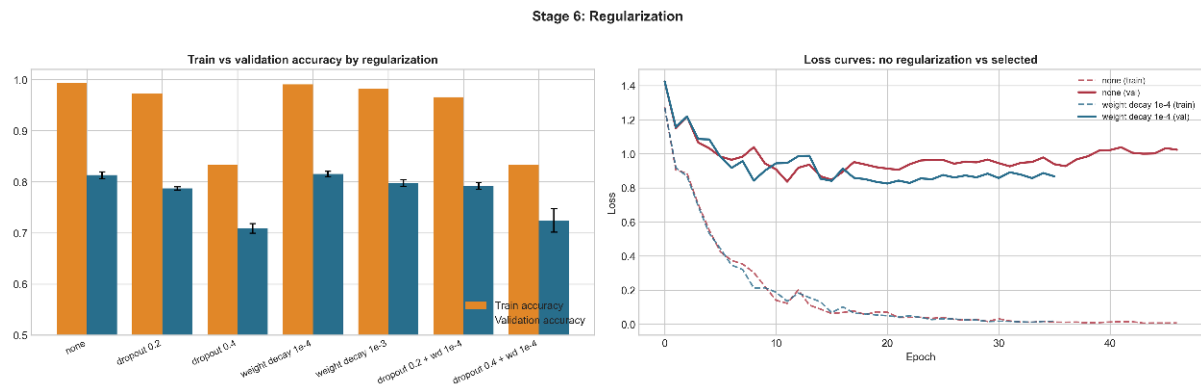


Figure 14: Stage 6: train and validation accuracy under different regularizers and loss curves for the unregularized versus selected model.

Inference: Dropout reduced the train/validation gap only by cutting training accuracy, and validation accuracy fell with it (70.9% at dropout 0.4), so it underfit rather than regularised. Weight decay of 10^{-4} gave a small gain (81.6%) without touching the fit. The loss curves show the overfitting pattern clearly: the validation loss bottoms out near epoch 20 and drifts upward while the training loss keeps falling towards zero.

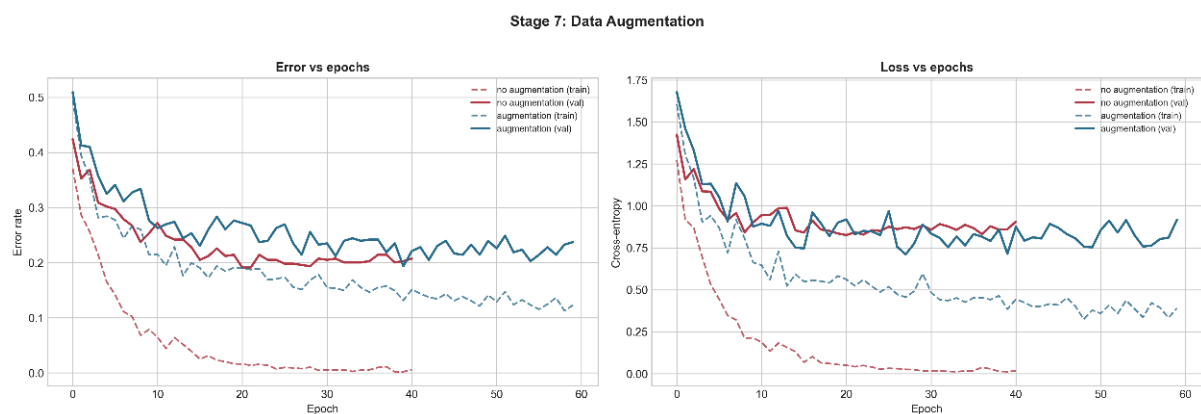


Figure 15: Stage 7: error and loss curves with and without per-epoch data augmentation.

Inference: Augmentation keeps the training error at 10% to 14% instead of 0% and holds the validation loss flat instead of rising, so it genuinely reduces overfitting. The validation accuracy, however, was 80.5% versus 81.6% without it, so it did not translate into better predictions on this data, where the remaining errors are ambiguous character pairs rather than over-specialised features.

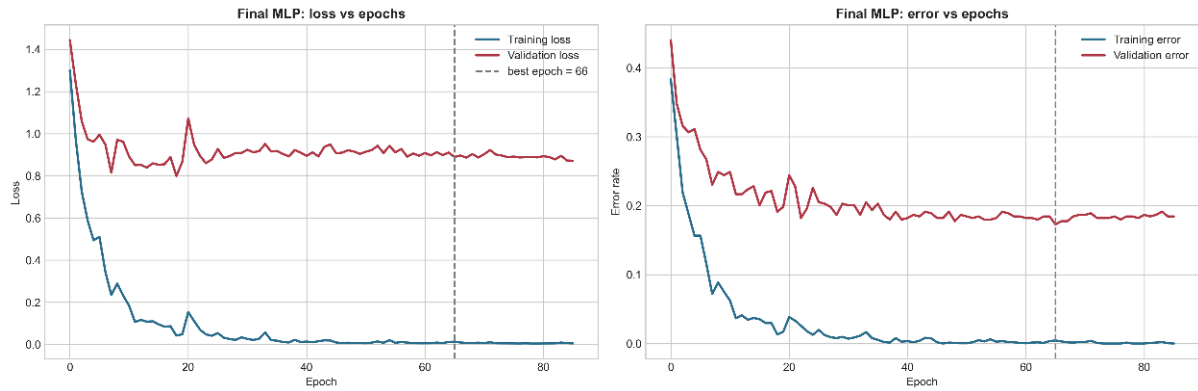


Figure 16: Final MLP: training and validation loss and error per epoch with the early-stopping point.

Inference: The training loss reaches 0.004 and the training error 0% by epoch 80, while the validation error plateaus at 17% to 20% from epoch 20 onwards and the validation loss stays near 0.9. The best validation epoch was 66 and training stopped at 86. The 3.8 s total training time shows how cheap the MLP is on this dataset.

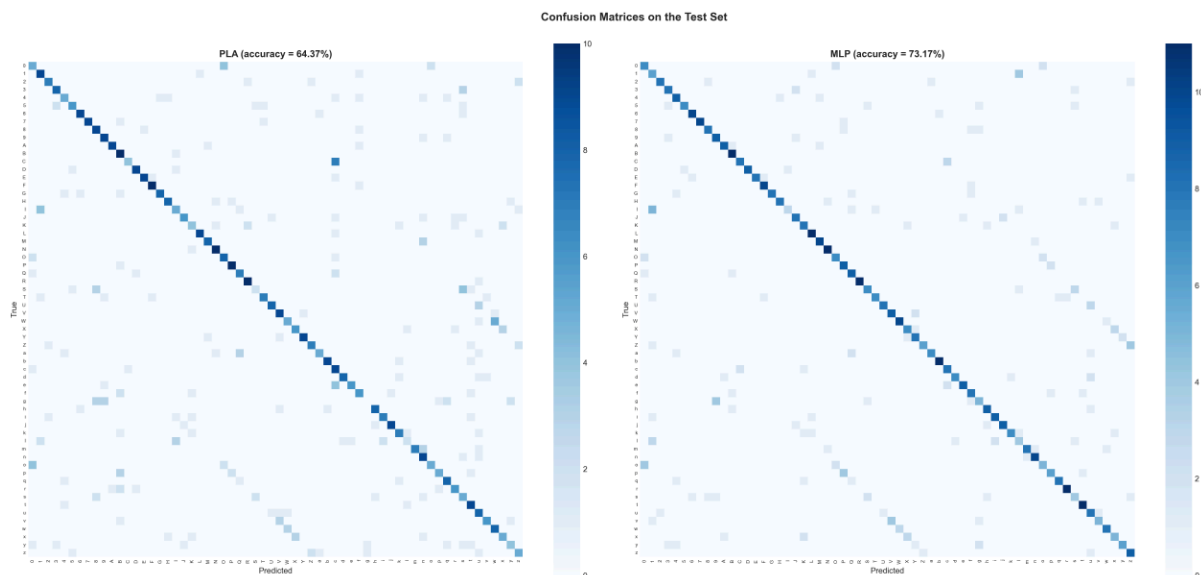


Figure 17: Confusion matrices of the PLA (left) and the tuned MLP (right) on the test set.

Inference: The MLP’s matrix has a darker diagonal and far fewer scattered off-diagonal cells, especially in the lowercase block at the bottom right where the PLA makes many random-looking errors. The off-diagonal cells that remain in both matrices sit at symmetric positions such as C/c, O/o/0, I/1/l, S/s, W/w and Z/z, which are the case and look-alike pairs identified in the EDA.

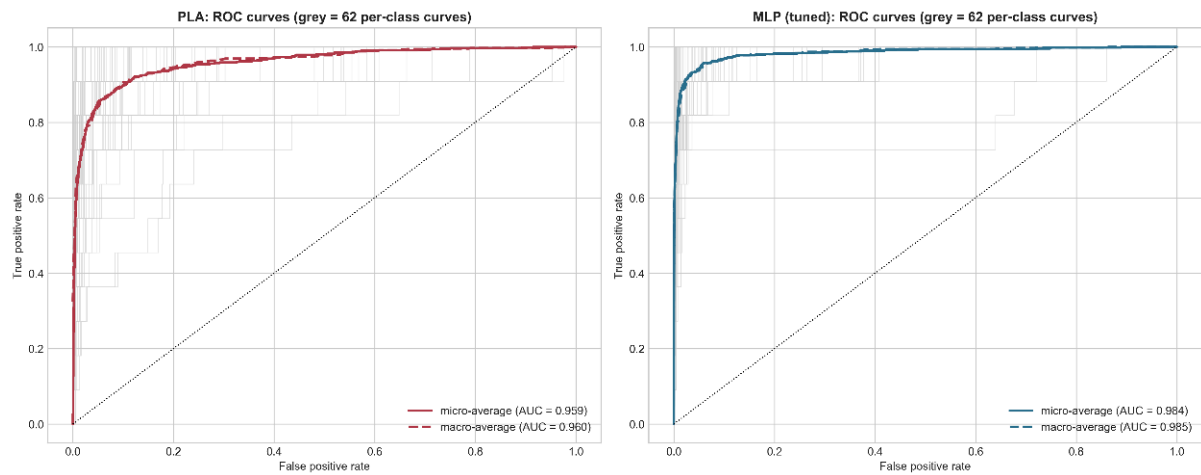


Figure 18: Micro and macro averaged ROC curves for the PLA and the MLP, with the 62 per-class curves in grey.

Inference: Both models rank the correct class highly most of the time, but the MLP's curves hug the top-left corner more tightly (micro AUC 0.984 versus 0.959) and its per-class curves are far more uniform, whereas the PLA has several classes whose curves sag well below the average. The ROC gap is smaller than the accuracy gap because the PLA's net inputs still rank the correct class second or third when its top choice is wrong.

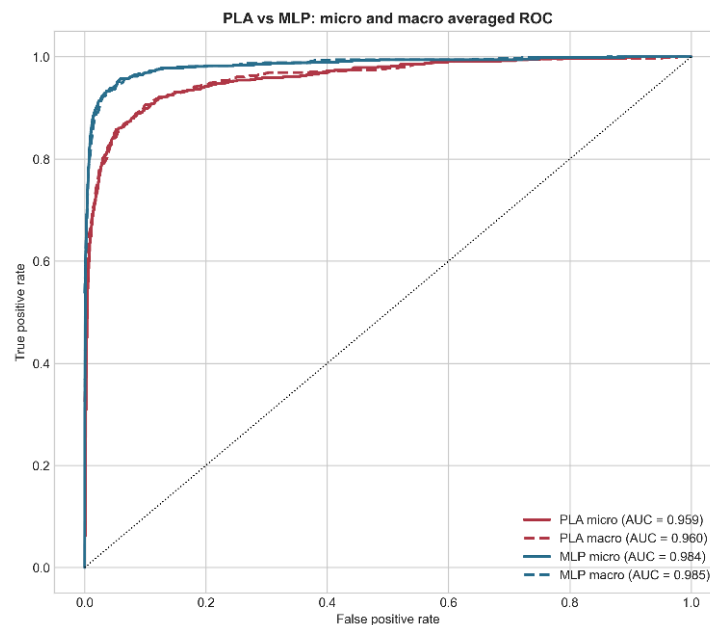


Figure 19: Micro and macro averaged ROC curves of both models on one plot.

Inference: Micro and macro averages coincide for each model because the classes are balanced. The MLP dominates the PLA at every operating point, with the largest gap at low false positive rates, where the MLP already recovers 80% of the positives at a 2% false positive rate.

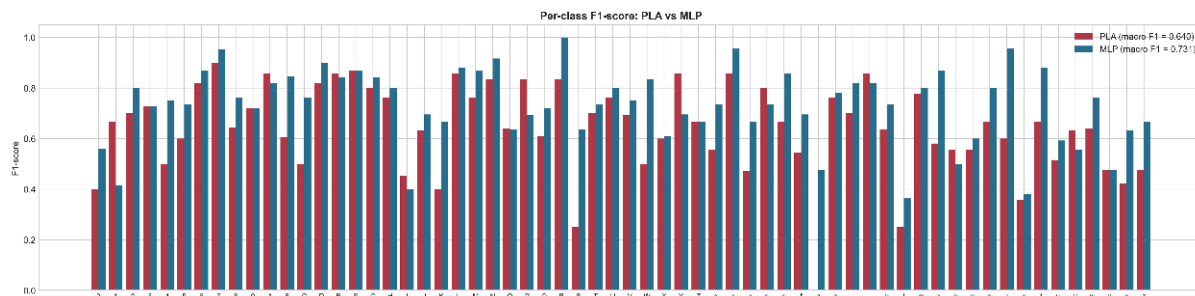


Figure 20: Per-class F1-score of the PLA and the MLP.

Inference: The MLP improves most on classes the PLA handles badly: g (F1 0.00 to 0.48), S (0.25 to 0.64), r (0.60 to 0.96), W (0.50 to 0.83) and n (0.58 to 0.87). The classes that stay hard for both are l (0.36), s (0.38), I (0.40), 1 (0.41), g, x, o and v, which are all members of ambiguous pairs.

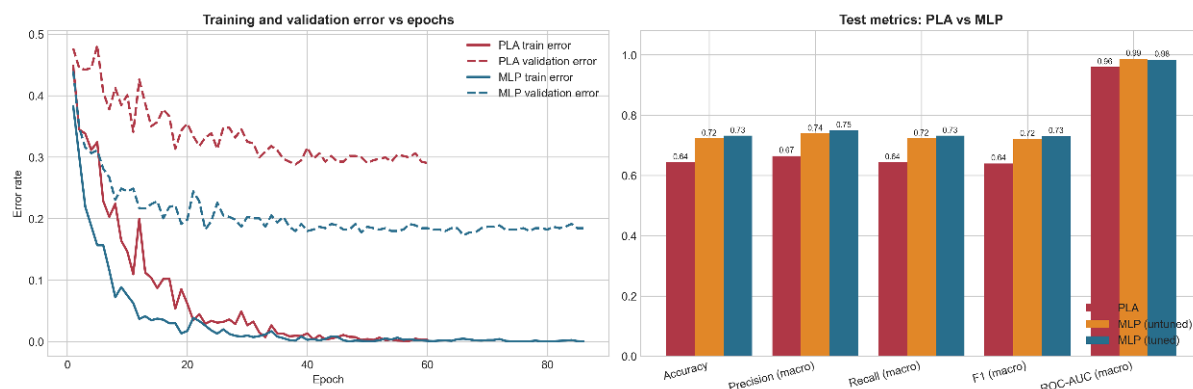


Figure 21: Convergence curves of both models (left) and test metrics of PLA, untuned MLP and tuned MLP (right).

Inference: The PLA’s validation error flattens at 29% while the MLP’s drops to 17% and does so within 20 epochs, so the MLP both converges faster and to a better solution. On the right, the untuned and tuned MLP bars are close together and both stand about 9 points above the PLA on accuracy, precision, recall and F1, which summarises the A/B result: the hidden layer is the decisive difference and tuning polishes it.

11 Discussion

11.1 Why does PLA underperform compared to MLP?

Because it is a linear model on a problem that is not linear. Each of its 62 units draws one hyperplane in pixel space, and the PCA scatter showed the classes sitting on top of each other there, so no set of hyperplanes separates them and the updates never stop (still 19 per epoch after 60 epochs). It did reach 99.1% training accuracy, since 2,294 points in 1,025 dimensions are almost separable, but that is memorisation: with no margin the hyperplanes hug the training points and fail on new handwriting, hence the 35-point train/test gap. The MLP’s hidden layer learns 256 stroke-like features and combines them non-linearly, which gives it curved boundaries and shared features, and its ROC-AUC of 0.985 versus 0.960 shows it ranks the right class higher even when its top guess is wrong.

11.2 Which hyperparameters had the most impact on MLP performance?

Batch size and learning rate together, by a wide margin: batch 16 at learning rate 0.05 with momentum blew up to 60.0% validation accuracy while batch 32 gave 81.3%. Optimizer and learning rate came next (72.96% for SGD at 0.01 up to 80.65% for momentum SGD at 0.05), then depth (81.3% with one hidden layer, 74.1% with three). Dropout mattered in the wrong direction and cost up to 10 points. The loss function (2.2 points) and the activation (0.8 points) barely mattered.

11.3 Did optimizer choice (SGD vs Adam) affect convergence?

Yes. Adam at 0.001 hit its best validation accuracy in about 22 epochs, plain SGD at 0.01 needed 52 and still ended 7 points lower, and SGD had to use a learning rate 30 times larger (0.3) to catch up. Adam was also much more forgiving about the learning rate (76.4% to 79.7% across a 50-fold range, against 73.0% to 80.2% for SGD). Momentum SGD at 0.05 turned out best overall (80.65%, only 0.2% spread across seeds, about 29 epochs), so momentum gave most of Adam's speed and, on this small dataset, slightly better generalisation.

11.4 Did adding more hidden layers always improve results? Why or why not?

No, it never did. Every deeper network was worse than a single 256-unit layer: (256, 128) 75.8%, (512, 256) 79.6%, (512, 256, 128) 74.1%, (256, 256, 256) 75.1%, while a wider single layer (512 or 1,024) changed nothing. With 2,294 training images the extra layers add parameters the data cannot pin down, and they were also harder to train: the deeper nets stopped at 87% to 91% training accuracy, so the learning rate tuned for one layer no longer suited them. One layer of stroke detectors is enough for a flattened 32×32 image; depth pays off with more data or with convolutions.

11.5 Did MLP show overfitting? How could it be mitigated?

Clearly: 99.6% training accuracy against 82.7% validation and 73.2% test, with the validation loss bottoming out around epoch 20 and creeping up while the training loss went to 0.004. Early stopping was used throughout and restores the best epoch. Weight decay of 10^{-4} helped a little. Dropout did not help at all, because with one 256-unit layer and 37 images per class it took away too much capacity and the model underfit. Augmentation shrank the train/validation gap from 17.5 to 5.8 points, so it did reduce overfitting, but validation accuracy stayed at 80.5% because the leftover mistakes are ambiguous case pairs that no regularizer can fix. What would really help is more data per class, a convolutional network that shrugs off small shifts and stroke changes, or features such as HOG instead of raw pixels.

11.6 Strengths and weaknesses of PLA vs MLP

The PLA is tiny (63,550 weights), trains in 3 s, has nothing to tune and its weights can be viewed as class templates, but it is stuck with straight boundaries, never converges here and generalises poorly. The MLP costs little more on this data (278,334 weights, 3.8 s), handles the non-linear structure and is 9 points more accurate, but it has many interacting hyperparameters, is sensitive to seed and learning rate, and overfits badly. Tuning mostly bought faster and steadier convergence rather than a big jump in accuracy.

12 Conclusion

A one-vs-rest Perceptron Learning Algorithm and a PyTorch Multilayer Perceptron were trained on the same 2,294 preprocessed 32×32 character images and compared on 682 held-out images. The PLA reached 64.37% test accuracy (macro F1 0.640, macro ROC-AUC 0.960) and never converged because the 62 classes are not linearly separable. Seven stages of tuning over three seeds each selected a single 256-unit ReLU layer, SGD with momentum 0.9 at learning rate 0.05, batch size 32, cross-entropy loss, weight decay 10^{-4} and early stopping; this MLP reached 73.17% test accuracy (macro F1 0.731, macro ROC-AUC 0.985), 8.8 points above the PLA, with the largest gains on lowercase letters. Batch size, learning rate and optimizer had the biggest influence on the MLP, deeper networks always hurt, Adam converged fastest but momentum SGD generalised best, and the MLP overfit strongly (99.6% train versus 73.2% test), which early stopping and weight decay only partly contained and data augmentation reduced without improving accuracy. The residual errors of both models are the visually identical case and look-alike pairs (0/O/o, 1/I/l, C/c, S/s, Z/z, W/w, V/v), which bound what any pixel-based classifier can achieve on this dataset.

13 References

- [1] F. Rosenblatt, “The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.
- [2] S. I. Gallant, “Perceptron-Based Learning Algorithms,” *IEEE Transactions on Neural Networks*, vol. 1, no. 2, pp. 179–191, 1990.
- [3] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning Representations by Back-Propagating Errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [4] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *Proc. 3rd Int. Conf. on Learning Representations (ICLR)*, 2015.
- [5] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.
- [6] T. E. de Campos, B. R. Babu, and M. Varma, “Character Recognition in Natural Images,” in *Proc. Int. Conf. on Computer Vision Theory and Applications (VISAPP)*, Lisbon, Portugal, 2009.
- [7] D. Dave, “English Handwritten Characters Dataset,” Kaggle, 2020. [Online]. Available: <https://www.kaggle.com/datasets/dhruvildave/english-handwritten-characters-dataset>
- [8] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems 32*, 2019, pp. 8024–8035.
- [9] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [10] T. Fawcett, “An Introduction to ROC Analysis,” *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.